

SIA 101 – Final Project

Integration Documentation

Member: Allyza Gwenn Suana

Mica-Ella Bacalso

Rixianne Jade Generalao

Progression Service Backend Integration

Table of Contents

1. System Overview
 2. Integration Objectives
 3. Integration Plan
 4. Integration Process (Step-by-Step)
 5. Challenges and Solutions
 6. Testing and Validation
 7. Architecture Diagrams
-

1. System Overview

1.1 Description of the Original Project

The original project is an academic learning management system developed in a previous subject. It provides basic course content delivery, lesson viewing, and quiz functionality. Students can access lessons, take quizzes, and view their scores.

1.2 Original Features

- User registration and login
- Lesson content viewing
- Quiz taking with score display
- Basic user profile management

- Course progress tracking (completion status only)

1.3 Existing Limitations and Problems

Limitation	Impact
No gamification system	Low student engagement and motivation
No XP or leveling system	Students cannot track skill growth
No leaderboard	No competitive element to encourage learning
No progress visualization	Users cannot see how close they are to next level
Basic quiz scoring only	No rewards or achievements for high scores
No cross-service communication	All logic tightly coupled in monolithic structure
No service separation	Difficult to scale or maintain individual features

2. Integration Objectives

2.1 Purpose of Each Integration

Progression Service Integration:

- Add gamification layer (XP, levels, leaderboards) to existing learning system
- Separate progression logic into independent microservice
- Enable real-time progress tracking across lessons and quizzes
- Provide competitive leaderboard for student engagement

2.2 Expected Improvements

Before Integration	After Integration
Static completion tracking	Dynamic XP-based progression
No motivation system	Level-up system with visual progress
Isolated user data	Cross-service user sync via webhooks
No rankings	Real-time leaderboard
Monolithic architecture	Microservice-oriented structure

2.3 Why the Integrations Were Necessary

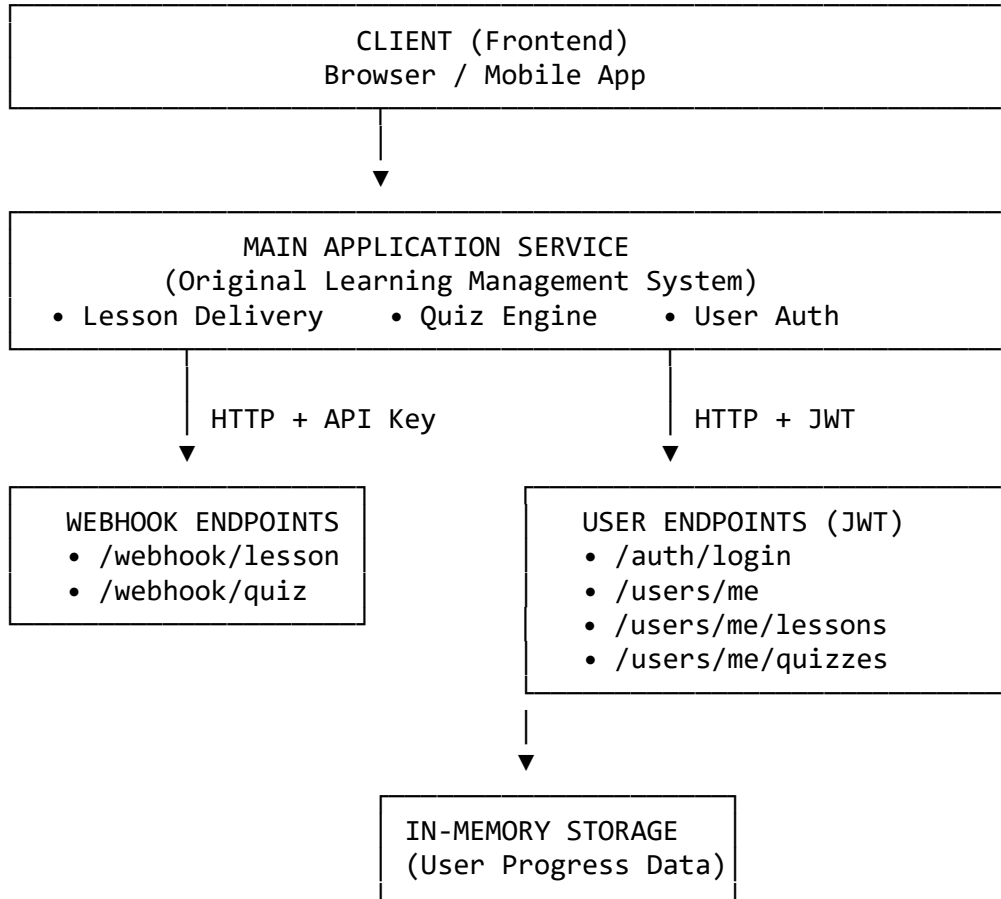
1. **Separation of Concerns:** Progression logic should not be mixed with content delivery
 2. **Scalability:** Independent scaling of progression features
 3. **Reusability:** The progression service can be reused across different projects
 4. **Maintainability:** Easier to update and debug isolated services
 5. **Industry Standard:** Microservice architecture is the modern backend standard
-

3. Integration Plan

3.1 List of Backend Services/APIs

Service Name	Type	Purpose	Auth
Progression Service	REST API	XP, levels, leaderboard	JWT + API Key
Health Check	Public Endpoint	Service status monitoring	None
Authentication	JWT Service	User token generation	None (public login)
Webhook Lesson	API Key Protected	External lesson completion	API Key
Webhook Quiz	API Key Protected	External quiz submission	API Key
User Profile	JWT Protected	Current user data	JWT
Admin User Lookup	API Key Protected	Cross-service user query	API Key
Leaderboard	Public	Ranked user listing	None

3.2 Planned Architecture



3.3 Service Responsibilities

Main Application Service:

- Content delivery and rendering
- Quiz question presentation
- User interface and experience
- Calls progression service via webhooks

Progression Service (This Integration):

- XP calculation and allocation
- Level progression management
- Leaderboard generation and ranking
- User progress persistence (in-memory)
- Authentication token generation

4. Integration Process (Step-by-Step)

4.1 Technologies Used

Technology	Purpose
Node.js	Runtime environment
Express.js	Web framework
JWT (jsonwebtoken)	User authentication tokens
CORS	Cross-origin resource sharing
In-Memory Map	Data storage (users)
Render	Cloud deployment platform

4.2 Installation and Setup Process

Step 1: Initialize Project

```
mkdir progression-service  
cd progression-service  
npm init -y
```

Step 2: Install Dependencies

```
npm install express cors jsonwebtoken
```

Step 3: Create Entry File (index.js)

- Set up Express server
- Configure CORS for cross-origin requests
- Implement middleware (JWT verification, API Key verification)
- Define route handlers
- Configure in-memory storage

Step 4: Environment Configuration

```
const PORT = process.env.PORT || 3000;  
const JWT_SECRET = "sia2024-progression-secret";  
const API_KEY = "main-system-integration-key";
```

4.3 Configuration Steps

Authentication Configuration:

- JWT tokens expire in 7 days
- Secret key hardcoded for demonstration (should use env vars in production)
- API key shared between main app and progression service

CORS Configuration:

```
app.use(cors()); // Allow all origins for development
```

XP System Configuration:

- Base XP for lesson completion: 50 XP
- Base XP for quiz: 30 XP
- Score bonus: up to 70 XP based on percentage
- Level scaling: exponential ($100 * 1.3^{(\text{level}-1)}$)

4.4 API Connection Process

Main App → Progression Service Communication:

1. User completes lesson in main app
2. Main app sends POST to /webhook/lesson-complete
3. Includes x-api-key header for authentication
4. Progression service records progress, calculates XP
5. Returns updated user data
6. Main app displays updated level/XP to user

Code Example (Main App Side):

```
// When user finishes Lesson
async function recordLessonCompletion(userId, username, lessonId) {
  const response = await fetch('https://sia101-final-project.onrender
.com/webhook/lesson-complete', {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
      'x-api-key': 'main-system-integration-key'
    },
    body: JSON.stringify({ userId, username, lessonId })
  });
  return response.json();
}
```

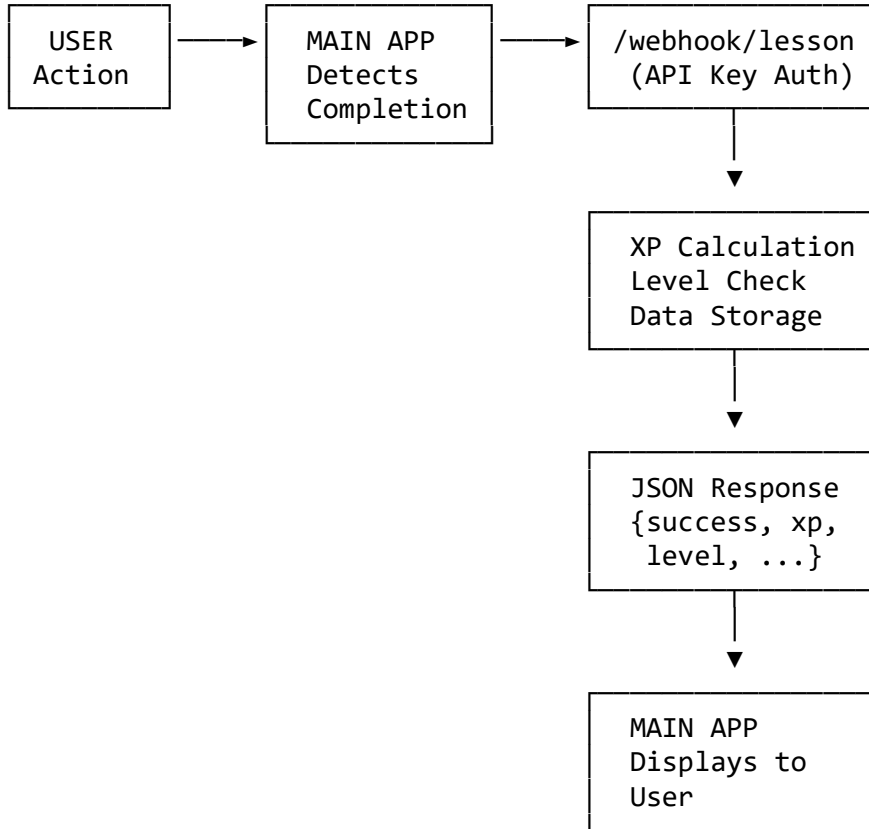
4.5 Deployment Setup

Render Deployment Steps:

1. Push code to GitHub repository
2. Connect Render to GitHub repository
3. Configure build settings: - Build Command: `npm install` - Start Command: `node index.js`
4. Set environment variables (PORT, NODE_ENV) 5. Deploy and verify health endpoint

Live Service URL: <https://sia101-final-project.onrender.com>

4.6 Backend Communication Workflow



5. Challenges and Solutions

5.1 Technical Issues Encountered

Challenge 1: CORS Errors

Problem: Browser blocked requests from main app to progression service

Solution: Added `cors()` middleware to Express server

Code: `app.use(cors());`

Challenge 2: Authentication Complexity

Problem: Needed two auth systems (JWT for users, API Key for services)

Solution: Implemented separate middleware functions

JWT Middleware: Extracts and verifies Bearer tokens

API Key Middleware: Validates shared secret key

Challenge 3: In-Memory Data Persistence

Problem: Data resets when server restarts (Render free tier)

Solution: Documented limitation; acceptable for academic demonstration -

Future: Can be upgraded to MongoDB/PostgreSQL

Challenge 4: XP Calculation Accuracy

Problem: Level progression with exponential scaling caused edge cases -

Solution: Implemented while-loop for multi-level jumps

Code:

```
while (xpAdded > 0) {
  const needed = getXpForLevel(user.level);
  if (user.xp + xpAdded >= needed) {
    xpAdded -= (needed - user.xp);
    user.xp = 0;
    user.level++;
    levelsGained++;
  } else {
    user.xp += xpAdded;
    xpAdded = 0;
  }
}
```

5.2 Deployment Problems

Challenge: Render Free Tier Cold Starts

Problem: Server takes time to wake up after inactivity

Solution: Added health check endpoint for monitoring

Workaround: Ping service before demonstration

5.3 Compatibility Issues

Challenge: Data Format Consistency - Problem: Main app and progression service had different user ID formats - **Solution:** Standardized on string-based userId across both services

6. Testing and Validation

6.1 API Testing Results

All endpoints tested and verified working:

Endpoint	Method	Status	Result
/health	GET	200 OK	Service running
/	GET	200 OK	Basic info
/auth/login	POST	200 OK	Token generated
/users/me	GET	200 OK	Profile returned
/users/me/lessons/complete	POST	200 OK	XP added
/users/me/quizzes/submit	POST	200 OK	Quiz scored
/webhook/lesson-complete	POST	200 OK	Progress recorded
/webhook/quiz-submit	POST	200 OK	Quiz recorded
/users/:userId	GET	200 OK	User data returned
/leaderboard	GET	200 OK	Rankings displayed

6.2 Postman Screenshots

[See attached Postman collection for full test cases]

Sample test flow:

1. Login → Receive JWT token
2. Complete lesson via webhook → Verify XP increase
3. Submit quiz via webhook → Verify score and bonus XP
4. Get user profile → Verify level progression
5. Check leaderboard → Verify ranking

6.3 Error Handling Verification

Error Case	Expected	Actual	Pass
Missing API Key	401 Invalid API key	401 Invalid API key	✓
Missing JWT	403 No token provided	403 No token provided	✓
Invalid JWT	401 Invalid token	401 Invalid token	✓
Missing required fields	400 error	400 error	✓
User not found	404 User not found	404 User not found	✓

6.4 Proof of Integration

Integration Test Sequence:

Test 1: Service Health

GET https://sia101-final-project.onrender.com/health
Response: {"status":"ok","service":"progression-service"}

Test 2: Webhook Lesson Completion

POST https://sia101-final-project.onrender.com/webhook/lesson-complete
Headers: x-api-key: main-system-integration-key
Body: {"userId":"123","username":"John","lessonId":"L1"}
Response: {"success":true,"xpGained":50,"level":1}

Test 3: User Progress Verification

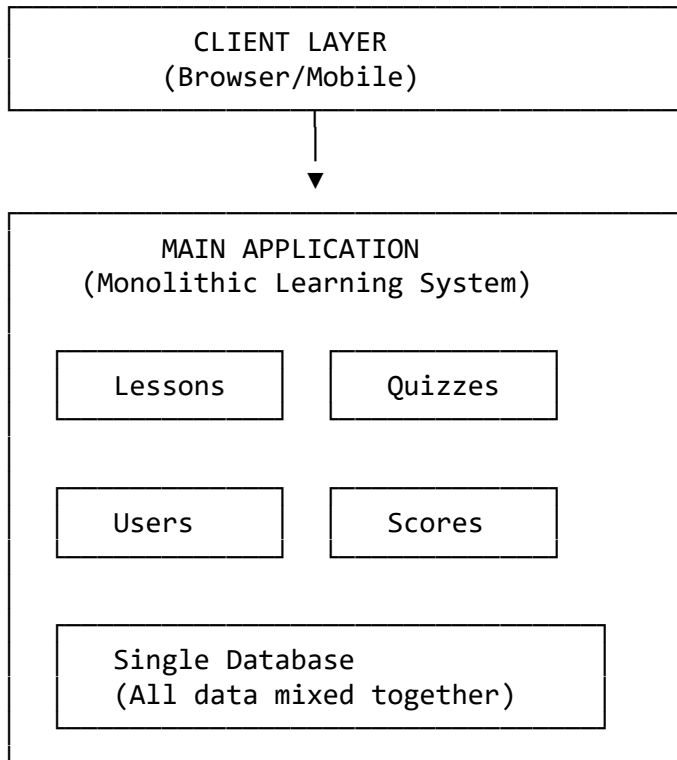
GET https://sia101-final-project.onrender.com/users/123
Headers: x-api-key: main-system-integration-key
Response: {"userId":"123","level":1,"xp":50,"totalLessons":1}

Test 4: Leaderboard Display

GET https://sia101-final-project.onrender.com/leaderboard
Response: {"leaderboard":[{"position":1,"username":"John","level":1,"xp":50}]}

7. Architecture Diagrams

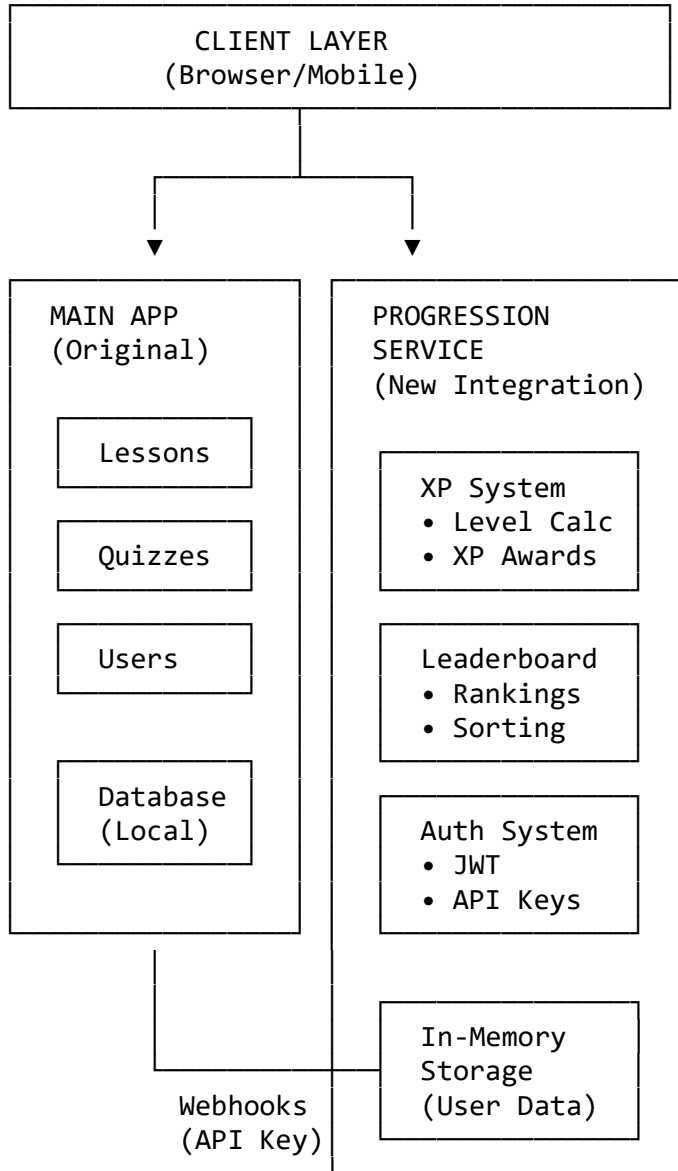
7.1 BEFORE Integration



LIMITATIONS:

- × No gamification
- × No separation of concerns
- × Hard to scale
- × No leaderboard
- × No XP tracking

7.2 AFTER Integration

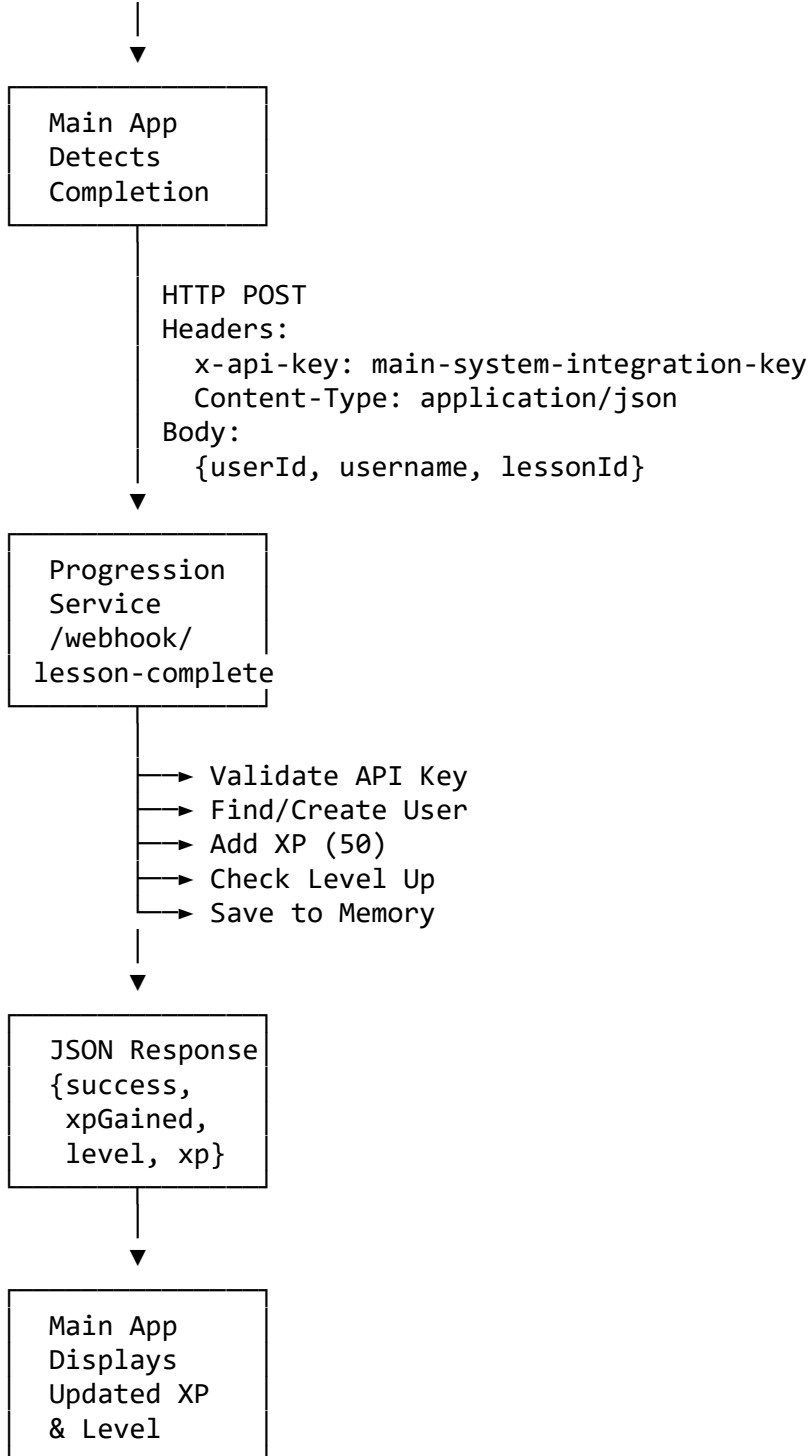


INTEGRATION POINTS:

- ✓ Webhook calls for lesson completion
- ✓ Webhook calls for quiz submission
- ✓ API key secured service-to-service
- ✓ JWT for user authentication
- ✓ Leaderboard accessible publicly

7.3 Data Flow Diagram

User Completes Lesson



Appendix A: GitHub Repository

Repository URL: <https://github.com/allyzagwennsuana202301889-bit/SIA101---Final-Project>

Project Structure:

```
progression-service/  
├── index.js           # Main server file  
├── package.json       # Dependencies  
├── README.md          # Setup instructions  
└── docs/  
    ├── integration-doc.md # This document  
    └── api-documentation.md # API reference
```

Appendix B: Deployment Links

Service	URL	Status
Progression Service	https://sia101-final-project.onrender.com	Live
Health Check	https://sia101-final-project.onrender.com/health	Active

Appendix C: Postman Collection

Collection Name: SIA101 Progression Service API

Export File:



SIA101_Progression_Service_Postman_Collection.json